

**Project Report
On
Multilingual Hate Speech Detection Using NLP**



**BENNETT
UNIVERSITY**
TIMES OF INDIA GROUP

**Session:
2025-26**

Faculty Mentor: Deep ti Sh a k y a

Associate Professor

**Submitted by:
DEVANSH**

**Natural Language Processing (CSET346)
Enrollment No. E23CSEU0940**

**SCHOOL OF COMPUTER SCIENCE ENGINEERING AND
TECHNOLOGY,
BENNETT UNIVERSITY**

ACKNOWLEDGEMENT

I would like to express my deepest and heartfelt gratitude to my faculty mentor, Deepti Shakya, for the guidance, continuous support, and feedback she has given to me throughout the duration of this research. Her expertise and encouragement helped me in shaping the direction of this study on Explainable Artificial Intelligence.

I also would like to thank the School of Computer Science Engineering and Technology at Bennett University for providing an excellent academic environment, the necessary resources, and the opportunity to undertake this project as part of the Special Topics in Computer Engineering course.

Finally, I am thankful to the department staff and the university administration for their underlying support, which facilitated the successful completion of this work.

Multilingual Hate Speech Detection Using NLP

Devansh
E23CSEU0940
NLP - CSET346,

Abstract—Hate speech on social media causes serious harm to various communities, particularly where users mix languages in a single post. This project delineates the comprehensive design of a web-based hate speech detection system encompassing the complete software stack: data collection and preprocessing, training of three machine learning models, a Flask web backend with user authentication, a PostgreSQL database, a browser-based frontend, and deployment on the Railway App. The system classifies text in English, Hindi, or mixed Hinglish as either Hate and Offensive (HOF) or Not Offensive (NOT). Three models are available for selection: a Logistic Regression classifier, a Bidirectional LSTM with attention head, and a MuRIL transformer. All models were trained on the amalgamated HASOC 2019 and HASOC 2020 datasets, comprising a total of 17,188 training samples. The superior model is MuRIL, which attained a macro F1 score of 0.8156 on the 3,948 sample test set. The implemented website facilitates user login, user registration, single-text prediction, batch prediction, prediction history tracking, and allows users to obtain a PDF of their results. **Keywords**—hate speech detection, multilingual natural language processing, Flask, PostgreSQL, Railway deployment, BiLSTM, MuRIL, HASOC, Hinglish, online application

Index Terms—hate speech detection, multilingual NLP, Flask, PostgreSQL, Railway deployment, BiLSTM, MuRIL, HASOC, Hinglish, web application

I. INTRODUCTION

One of the most noticeable issues on social media platforms these days is hate speech. Posts that are offensive, hateful, or vulgar hurt the target's mental health. [1], [2]

both persons and organizations, and occasionally they are involved in actual acts of violence. Millions of posts are published every hour, making large-scale manual moderation impractical. For this reason, automated detection methods are required. [1], [2]

The problem is more challenging in India than in most other countries since Indian users sometimes blend Hindi and English in a single statement. This kind of writing, known as Hinglish or code-switching, defeats most popular natural language processing systems designed for a single language.

A system that cannot manage these variations may miss most of the problematic content when a word like "bakwas" (nonsense) appears in ten different spellings on the same platform.

This project executes a system that tackles this problem. The work consists of:

- A single preprocessing pipeline for English, Hindi (in Devanagari script), and Hinglish text preparation.
- Three models were trained.
- A Flask web application backend with user authentication, SQLAlchemy and three prediction API routes..

- A simple yet beautiful HTML, CSS and JavaScript frontend with multiple pages that has a dashboard, history page, batch mode, model comparison, and PDF export.

- A PostgreSQL database with environment-based configuration and automatic table creation hosted on Railway.

- Complete deployment with a Procfile-based gunicorn server on the Railway cloud platform.

In addition to creating accurate models, the project aims to put them into a functional web service that non-technical users can use from a browser.

II. RELATED WORK

A. Early Approaches

As social media platforms growing are growing there is also increase in hate speech due to which, hate speech detection has been framed as a text classification challenge. Waseem and Hovy [1] were among the first to provide a labeled Twitter dataset and showed that character n-grams in combination with Logistic Regression provide a decent baseline. Davidson et al. [2] made an improvement by differentiating between hate speech and just offensive language, demonstrating that the two are easily conflated. Nobata et al. [3] showed that the combination of linguistic variables (n-grams, dependency routes) and comment metadata can significantly improve the detection of abusive content in Yahoo comments. Schmidt and Wiegand [4] provided a detailed review of feature engineering techniques, showing that lexicon-based, syntactic and distributional features provide orthogonal signal.

B. Deep Learning and Attention

Neural methods learned representations from text directly, replacing hand-crafted features. The attention mechanism was first proposed in [5] for sequence-to-sequence tasks, and then used to classification tasks to concentrate on the most discriminative terms. Zhang et al. [6] showed the benefit of integrating character and word-level CNN encoders with attention to improve hate speech identification over single-granularity models. Hochreiter and Schmidhuber [7] proposed the LSTM cell as the base of BiLSTM classifiers, the bidirectional version [8] reads sequences both ways and has become the standard in text categorization. Bojanowski et al. [9] proposed FastText, which expresses words as bags of character n-grams, thereby allowing embeddings to be constructed for out-of-vocabulary words, an important trait for code-mixed social media material with non-standard spelling.

C. Transformer Models

BERT was proposed by Devlin et al. [10], which pre-trains a deep bidirectional transformer on masked language modeling and next-sentence prediction. Fine-tuning BERT on downstream tasks consistently surpassed past state-of-the-art across numerous NLP benchmarks. Mozafari et al. [11] have directly utilized BERT for hate speech recognition and shown considerable improvements over LSTM based systems on benchmark datasets. Liu et al. [12] proposed RoBERTa by omitting next-sentence prediction and training with larger batches, which greatly improved transfer performance. XLM-RoBERTa, a cross-lingual model trained on 100 languages, was provided by Conneau et al. [13] and has become the preferred multilingual baseline for low-resource and code-mixed applications.

D. Multilingual Detection

One particular concern is code-switching, the use of two languages in one utterance. Sitaram et al. [14] explore the NLP difficulties of Hinglish (Hindi–English code-mixing), and find that most traditional NLP pipelines fail, as they presuppose a single language. Patro et al. [15] investigated abusive language identification in Hinglish and proved that subword tokenization with transformer models is better than word-level approaches. Kula et al. [16] demonstrated that multilingual BERT performs much better than monolingual models for hate-speech tasks in low-resource settings. MuRIL [17] is a transformer pre-trained on 17 Indian languages and their transliterated versions. MuRIL outperformed multilingual BERT and XLM-R on a series of Indian language benchmarks including code-mixed tasks. MuRIL was also used by Velankar et al. [18] on Marathi hate speech where they showed its superiority over XLM-R for Indic text..

E. HASOC

The standard benchmarks utilized in this work are provided by the FIRE conference’s HASOC (Hate Speech and Offensive Content) shared tasks. Mandl et al. [19] curated the 2019 edition comprising English and Hindi social media postings with HOFNOT labels; the top systems employed fine-tuned multilingual BERT variations. The 2020 dataset [20] was augmented with a Marathi track. Ranasinghe et al. [21] presented one of the best findings in HASOC 2019 by ensembling fine-tuned BERT models and proved the importance of cross-lingual transfer for low-resource Hindi recognition.

F. Web Applications

There are problems such as latency, scalability and usability when deploying NLP models in commercial web applications. This project uses recommended practices for constructing Python online services with Flask by Grinberg [22] such as application factory pattern, blueprint-based routing, and SQLAlchemy ORM integration. With the advent of cloud-native deployment of ML models through platforms like Railway and Heroku, it is now possible to launch a whole NLP service stack from a single GitHub repo without having to manage server infrastructure.

III. DATASET

A. Source

This project uses data from HASOC 2019 and HASOC 2020 [19], [20]. Each edition provides train and test splits in English and Hindi.

The task is binary classification where each label each post as HOF (Hate, Offensive) or NOT (not offensive). All training data from 2019 and 2020 was merged as a single training data to give model as much data possible for training

B. Statistics

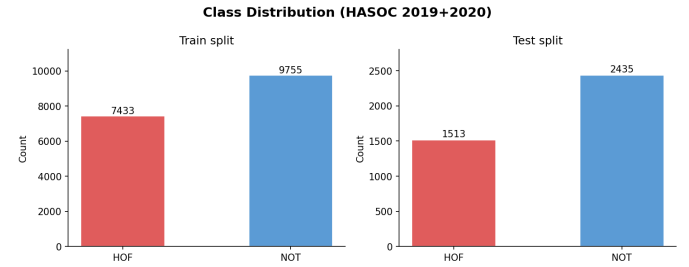


Fig. 1. Distribution of lables for Train and Test Set.

The training set has 17,188 samples: 7,433 HOF and 9,755 NOT. The combined test set has 3,948 posts: 1,513 HOF and 2,435 NOT. Table I shows the breakdown by year and language.

. Table I shows the breakdown by year and language.

TABLE I
DATASET STATISTICS — HASOC 2019 + 2020 COMBINED

Year	Lang.	Total	HOF	NOT
<i>Training set</i>				
2019	English	5,852	2,261	3,591
2019	Hindi	4,665	2,469	2,196
2020	English	3,708	1,856	1,852
2020	Hindi	2,963	847	2,116
Total		17,188	7,433	9,755
<i>Test set</i>				
2019	English	1,153	288	865
2019	Hindi	1,318	605	713
2020	English	814	423	391
2020	Hindi	663	197	466
Total		3,948	1,513	2,435

C. Class Imbalance

NOT posts outnumber HOF posts by roughly 1.3 to 1 in the overall training set. The imbalance is more severe in some individual splits; for example, the 2020 Hindi training data has 2.5 times as many NOT posts as HOF posts. All three models address this explicitly as described in Section V.

IV. PREPROCESSING

A good preprocessing pipeline is essential for any kind of AI model; without this the predictions become useless and the model may not function for its original purpose. For social media text in a multilingual setting we apply a shared set of cleaning steps to every post, followed by small model-specific adjustments.

A. Shared Cleaning Steps

Every training data goes through the following steps regardless of which model will process it:

- 1) **URL removal:** Web links are replaced with a generic URL token. The destination of a link carries no hate-speech signal in the text itself.
- 2) **Emoji Removals, Number Removals:** Emojis and numbers are removed from the text as it does contain any contextual meaning.
- 3) **Hashtags:** The hash symbol is removed but the word following it is kept, since hashtag words often carry meaning.
- 4) **Character Repeated:** SCharacter Repeated: Strings of repeated characters are compressed (e.g., “goood” becomes “good”)
- 5) **Whitespace normalisation:** Multiple spaces and invisible control characters are collapsed to a single space.

B. Hinglish Normalisation

Transliteration Normalization for Hinglish / code-mixed text is done. Normalisation is one of the most significant preprocessing procedures. Hindi words in English letters can be spelled differently and there is no standard spelling, hence the same word can be found in a myriad of forms in the dataset. For example, an insulting Hindi term could be spelled as “ganda”, “gandaa”, “gandha” or “gndaa” in different posts. Without normalization, each spelling is viewed as a new term, and each spelling is too infrequent for the model to acquire a valid relationship. For this we use a fuzzy-matching normalisation step. A manual mapping from Hinglish to English is done with similar meanings. For each word token in a post, we calculate its similarity to each dictionary key using edit distance. If the closest match is sufficiently similar (over a threshold of 0.85 similarity), the token is replaced with the canonical form. This step precedes all other preprocessing and is enforced on all three models.

C. Model-Specific Adjustments

After the common steps, each model gets a slightly different version of the text:

a) **Logistic Regression:** No additional tokenisation is required. The text is supplied directly to the TF-IDF vectoriser which performs the splitting inside.

b) **BiLSTM:** The text is tokenized by a script-aware rule which handles Devanagari characters (used for Hindi) and Latin characters (used for English and Hinglish) properly. A little extra step removes a certain Hindi diacritic, which makes the same word appear as two different tokens depending on

the way it was typed. For lengthier words, prefix and suffix tokens are included to give the model sub-word information, which is useful for morphologically rich languages like Hindi

c) **MuRIL:** No manual tokenisation is required. The cleaned text is then tokenised directly with the MuRIL tokeniser, which automatically handles all languages and scripts. Sequences are trimmed to a maximum of 64 tokens.

V. MACHINE LEARNING MODELS

A. Model 1: Logistic Regression

The simplest of the three models is Logistic Regression. It works by modeling each post as a numerical feature vector and then learning a set of weights to distinguish HOF posts from NOT posts. The features we employ are TF-IDF values, which is a measure of how often a word occurs in a post compared to how often it occurs across all posts. Rare but discriminative words (e.g. offensive slurs) have high TF-IDF values; common words (e.g. “the” or “is”) have low values. We calculate two sets of features: word-level features (single words and two-word phrases), and character-level features (sequences of 3 or 4 consecutive characters). The character properties are especially important as they overlap across different spellings of the same word. Both sets are concatenated to one huge feature vector per post. To account for the fact that HOF posts are less frequent than NOT posts, the model is configured to provide equal overall importance to both classes during training. The regularisation strength (preventing overfitting) was found by testing five different values with 5-fold cross-validation; the optimum value was $C = 1.0$.

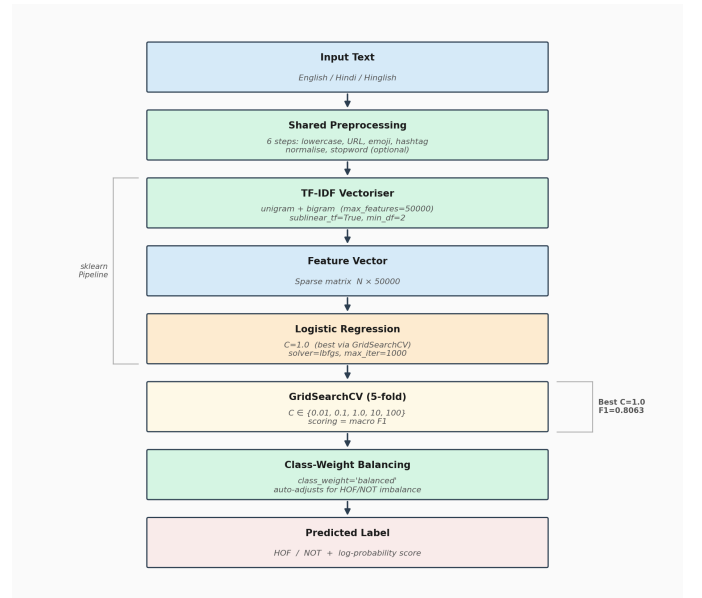


Fig. 2. LR architecture: TF-IDF vectorisation and logistic regression.

B. Model 2: Bidirectional LSTM with Attention

Bidirectional LSTM (BiLSTM) is a type of neural network. “Bidirectional” implies it reads the message both ways

(left to right and right to left) at the same time ,thus every word gets context on both sides of it.

We used an attention mechanism on top of BiLSTM for the output [5].

After the whole text is read, attention assigns a score to each word location describing how relevant this word is for the final assessment. Then the model summarizes the message by computing a weighted average over all word positions, with more weight assigned to the most relevant terms. This is especially useful for hate speech, since a single offensive term in an otherwise neutral post should dominate the prediction.

Word embeddings are initialized from FastText pretrained vectors [9] trained on huge web crawls in English and Hindi. This gives the model a head-start in understanding word meaning rather than having to learn everything from the 17K training samples.

The embedding layer is frozen for the first two training epochs until the rest of the network stabilizes and then unfrozen so that the vectors can be changed to the domain of hate-speech.

To tackle the problem of the class imbalance, the loss function adds an additional penalty for missed HOF predictions. Early stopping mechanism is also used for 5 consecutive epochs meaning if validation loss and validation accuracy does not improve model is stopped.

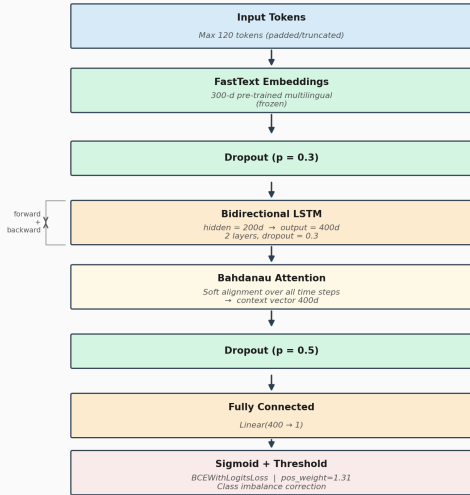


Fig. 3. BiLSTM with Bahdanau attention architecture.

C. Model 3: MuRIL Transformer

MuRIL is a transformer model developed by Google Research [17]. It is pre-trained on vast quantities of text in 17 Indian languages including transliterated and code-mixed content, making it directly suitable to the Hindi-English-Hinglish content in the HASOC dataset. The model architecture is based on BERT [10], which includes 12 transformer layers with self-attention mechanisms that may relate each word to each other word in the sequence. This offers it a far deeper

sense of context than either LR or BiLSTM. We next add a single classification layer on top of the model’s summary representation, and fine-tune the whole network end-to-end, on the training data. Fine-tuning uses a very tiny learning rate to avoid overwriting the pre-trained knowledge and a warmup schedule to slowly boost the learning rate in the first few training steps. The class weights handle the imbalance between HOF and NOT. The best saved model was at epoch 2, which shows how fast MuRIL adjusts due to its pre-training.

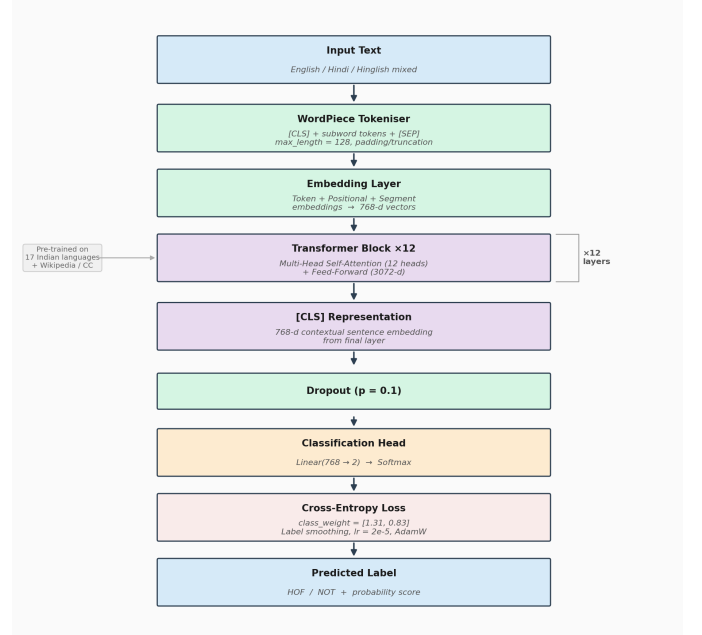


Fig. 4. MuRIL fine-tuned transformer architecture.

VI. RESULTS

A. Overall Performance

All three models after training are then evaluated on the same 3,948-sample test set using macro-averaged F1 score as the primary metric. Macro F1 treats both classes equally and is the standard metric for HASOC. Table II shows the results.

TABLE II
TEST SET PERFORMANCE — HASOC 2019+2020 COMBINED

Model	Accuracy	Macro F1
Logistic Regression	0.8176	0.8063
BiLSTM + Attention	0.8050	0.7890
MuRIL (fine-tuned)	0.8273	0.8156

MuRIL achieves the best result on both accuracy and macro F1. Logistic Regression performs surprisingly well, staying within 0.010 macro F1 of MuRIL and outperforming the BiLSTM. The BiLSTM achieves the lowest score primarily because it misses many HOF posts (low HOF recall). Fig. 6 illustrates the accuracy and macro F1 for all three models.



Fig. 5. Training and validation curves for BiLSTM and MuRIL. LR is omitted as it has no epoch-based training history.

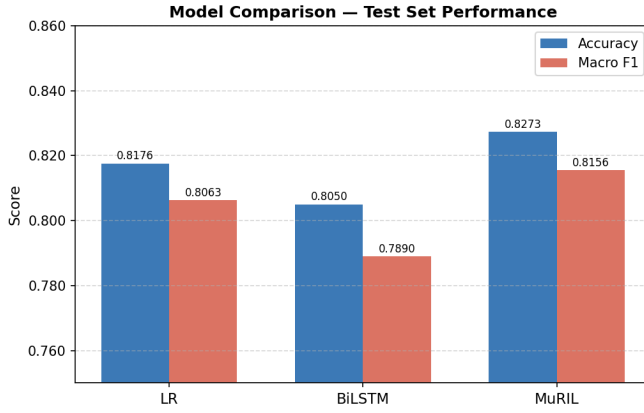


Fig. 6. Test set accuracy and macro F1 for all three models. MuRIL achieves the highest macro F1 score, while Logistic Regression performs competitively despite its simplicity. BiLSTM shows lower performance mainly due to reduced recall for the HOF class.

B. Per-Class Results

Table III shows precision, recall, and F1 for each class. The most important observation is HOF recall: the BiLSTM detects only 69.1% of all actual HOF posts, compared to 75.3% for LR and 75.1% for MuRIL. All three models detect NOT posts more reliably than HOF posts because NOT is the majority class with more training examples.

C. Comparison with Prior Work

To evaluate the effectiveness of the proposed models, we compare them with existing approaches across English and Indic datasets. Since prior work is evaluated on different datasets and tasks (binary vs. multi-class), the comparison is indicative rather than strictly direct.

The results show that transformer-based models consistently outperform traditional and deep learning approaches. The proposed MuRIL model achieves strong performance across

TABLE III
PER-CLASS PRECISION (P), RECALL (R), AND F1

Model	Class	P	R	F1
LR	HOF	0.768	0.753	0.759
	NOT	0.848	0.859	0.852
BiLSTM	HOF	0.775	0.691	0.731
	NOT	0.820	0.876	0.847
MuRIL	HOF	0.788	0.751	0.769
	NOT	0.850	0.874	0.862

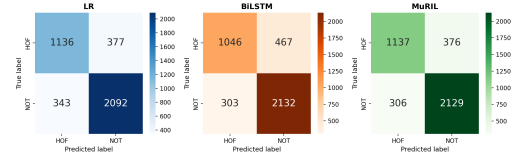


Fig. 7. Confusion matrix — ALL model (test set, 3,948 samples) It can be observed that most misclassifications occur in the HOF class, indicating that detecting offensive content is more challenging than identifying non-offensive text. This is primarily due to class imbalance and linguistic variability in Hinglish text..

both English and Hindi datasets, demonstrating robustness in multilingual and code-mixed settings.

From the results in Figs. 8 and 9, the following observations can be made:

- Transformer-based models consistently outperform classical and deep learning approaches across both English and Indic settings.
- The proposed MuRIL model achieves the best performance on the HASOC-based multilingual dataset and shows strong results across both English and Hindi.
- While large-scale transformer models (e.g., BERT) achieve higher scores on larger English datasets, the proposed approach remains competitive despite being trained on a smaller, code-mixed dataset.

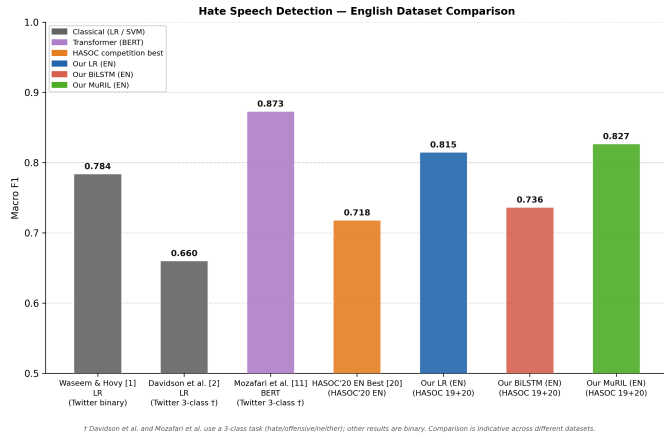


Fig. 8. Comparison of hate speech detection models on English datasets. Classical models (Logistic Regression) and transformer-based approaches (BERT) are compared with the proposed models. The proposed MuRIL model achieves competitive performance (Macro F1 = 0.8266), outperforming traditional baselines and HASOC competition results. *Note: Some prior works use a 3-class classification setup (+), while others use binary classification; comparisons are therefore indicative.*

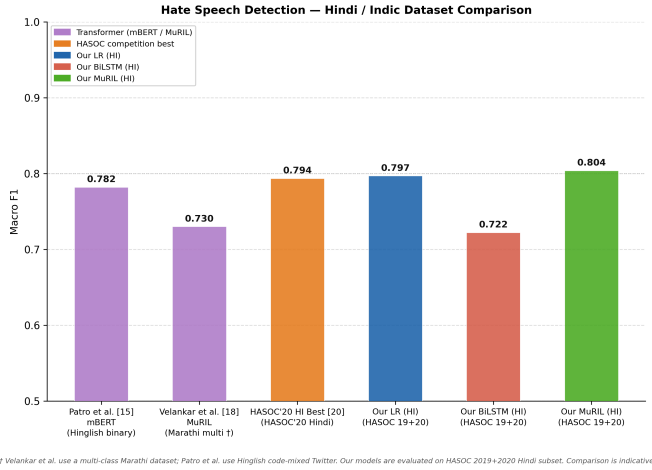


Fig. 9. Performance comparison on Hindi and code-mixed (Indic) datasets. The proposed MuRIL model achieves the highest performance among compared Indic models (Macro F1 = 0.8037), outperforming approaches such as mBERT [15] and the HASOC 2020 competition best system [20]. This highlights the effectiveness of transformer-based models for multilingual and code-mixed text. *Note: Velankar et al. [18] use a multi-class Marathi dataset, and Patro et al. [15] use Hinglish Twitter data; comparisons are indicative.*

- Logistic Regression performs competitively despite its simplicity, making it a strong lightweight baseline.
- BiLSTM shows comparatively lower performance, especially on Hindi data, due to challenges in handling noisy and code-mixed text.

VII. SYSTEM PIPELINE

The machine learning models are integrated in a whole web application. This section covers the overall system design, including backend, frontend, database and deployment.

A. Overview

It is a three tier architecture pipeline and it includes:

- 1) **Frontend:** Basic and interactive UI is developed using HTML, CSS and JS.
- 2) **Backend:** Flask app that takes http requests, loads the ML models, runs predictions, reads/writes the database.
- 3) **Database:** A PostgreSQL database (on Railway) or a local SQLite file (for development) that contains user accounts and forecast history.

Fig. 11 The following figure show the flow, first the user submits the text using a web based application then it reaches to Flask backend using API endpoints then they call the required model to produce a prediction, the results gets saved to database

B. Backend

The backend is a Python application built with the Flask micro-framework. We chose Flask as it is lightweight, easy to combine with PyTorch and scikit-learn, and widely supported by cloud deployment platforms. The program uses application factory pattern. All extensions (database, login manager) are initialized inside a create_app() function instead of module level. It makes the application easy to test and to deploy. The code is divided into three route blueprints:

- **Auth blueprint (/auth/...):** user registration, login and logout. Passwords are stored as bcrypt hashes. Flask-Login handles the session cookie so that you cannot visit sites that are protected without logging in.
- **Main Blueprint (/dashboard, /batch, /compare, /history):** controls main prediction pages. When the user enters text, the backend calls utils/predict.py which loads the chosen model, runs the preprocessing pipeline and produces a prediction dictionary including the label (HOF or NOT) and a confidence score. Then the result is saved in the database.
- **Report blueprint (/report/...):** Produces a downloadable PDF report on the user's forecast history. The PDF is generated in memory using the ReportLab package and contains a summary table, a pie chart showing the proportions of HOF vs NOT, and details for each prediction. The prediction utility works with all three model backends.

The correct model is lazy loaded (on first usage, not on startup) to reduce memory burden. Each backend has its own pre-processing pipeline to ensure that the text transformation is compatible with what was utilized during training.

C. Frontend

The frontend consists of multiple HTML pages served by Flask's Jinja2 templating engine. A shared base template provides the navigation bar and page layout so that every page has a consistent look. The individual pages are:

- **Home page:** A public landing page describing the system with a call-to-action to register or log in.
- **Register / Login pages:** Forms for account creation and authentication. Client-side validation gives immediate feedback before the form is submitted.
- **Dashboard:** The main prediction page. The user types or pastes text, selects one of the three models from a

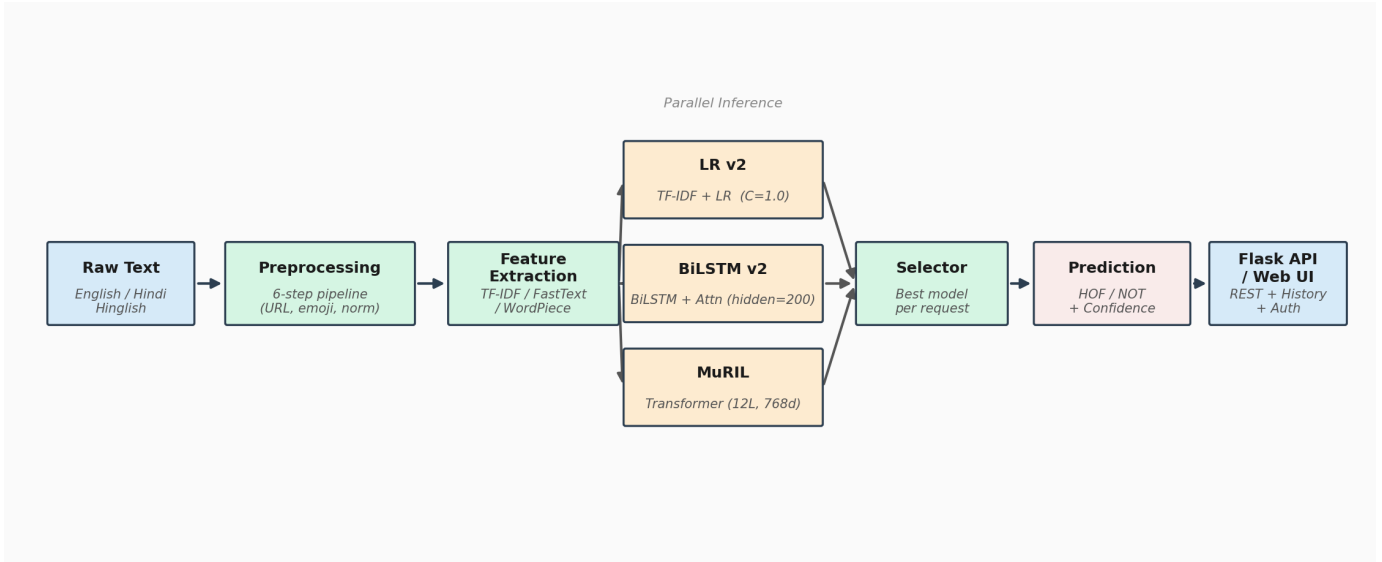


Fig. 10. End-to-end hate speech detection system prediction pipeline.

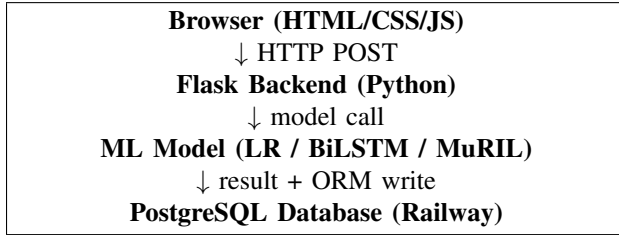


Fig. 11. System architecture overview

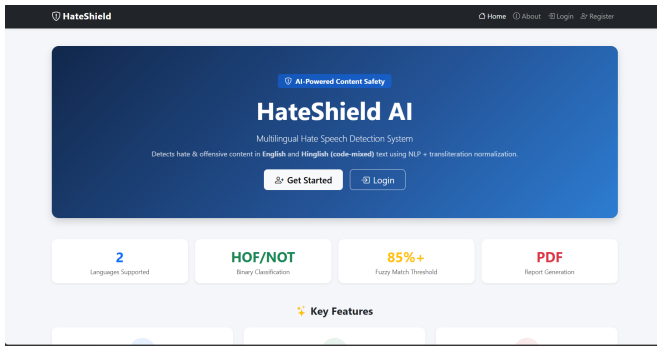


Fig. 12. HateShield AI web interface showing the landing page with multilingual hate speech detection features.

dropdown, and submits. The result (HOF or NOT, with confidence percentage) appears on the same page without a full reload using a JavaScript fetch call.

- **Batch prediction page:** Allows the user to upload a CSV file containing many posts. The backend processes each row in sequence and returns a results table with predictions and confidence scores for every row. Results can be downloaded as a new CSV.
- **Compare page:** Runs the same input text through all three models simultaneously and displays the results side

by side, so the user can compare their outputs.

- **History page:** Shows a table of all predictions made by the current user, sorted by time. The user can filter by model type, delete individual records, or download a PDF report.
- **About page:** Explains the project, the models, and the dataset to non-technical users.

All pages are styled with a consistent CSS stylesheet that uses a blue and white colour scheme. The layout is responsive so that the application is usable on mobile devices as well as desktop browsers.

D. Database

The application uses SQL Alchemy which lets the backend interact with the database using Python. It has two tables in it which are:

- **User:** user ID, username, email address, and password are stored in this. Each user has a one-to-many relationship with predictions.
- **Prediction:** stores the user ID (foreign key), the original input text, the predicted label (HOF or NOT), the confidence score, and a timestamp. The normalised text is also stored when Hinglish normalisation was applied.

Database tables are created automatically on first startup by calling `db.create_all()` inside the application context. This means the application can start on a fresh PostgreSQL instance without any manual setup.

In production, the database connection string is provided through the `DATABASE_URL` environment variable set by Railway. Locally, the application falls back to a SQLite file so that developers can run the application without installing PostgreSQL.

VIII. DEPLOYMENT

A. Deployment Platform: Railway

Railway is a modern cloud platform that can deploy any application from a GitHub repository without requiring the developer to manage servers, containers, or infrastructure. It supports Python, detects the application type automatically from the repository contents, and provides a managed PostgreSQL database as an add-on service. Railway was chosen for this project because of its simplicity: connecting a GitHub repository, adding the database plugin, and setting environment variables is sufficient to get the application running on a public URL.

B. Web Server

Web server Flask's built-in development server is not ready for production because it's single-threaded and not meant to handle many concurrent queries. In production the application is served via gunicorn, a Python WSGI server for production. A Procfile at the root of the project instructs Railway which command to run: `web: gunicorn wsgi:application wsgi.py` instantiates the Flask app using the application factory and exposes it as application so that gunicorn may import and serve it. Gunicorn forks off certain worker processes, so the server can process multiple requests at once.

```
web: gunicorn wsgi:application
```

The `wsgi.py` code generates the Flask application using the application factory and exposes it as application so that gunicorn may import and serve it. Gunicorn launches many worker processes, so the server can handle multiple requests in concurrently.

C. Environment Variables

Never hard code secret configuration settings into the source code. They are defined as environment variables in the Railway dashboard instead:

- `SECRET_KEY`: A lengthy random string that Flask uses to sign session cookies. Without this, an attacker may create a fake login cookie.
- `DATABASE_URL`: This is the entire connection string for PostgreSQL that Railway automatically gives you when you add the database plugin.

D. Model Storage

Three trained models were saved (Logistic Regression, BiLSTM PyTorch, and MuRIL) in the `model3/` directory and commit them to the repository. When Railway releases a new version it clones the complete repository including the model files, thus the models are available right away without any further download step.

E. Deployment Steps

The complete process for deploying a functioning local application to a live public URL requires the following steps:

- 1) Put all code and model files in a Github repo.

- 2) Create a new project in Railway and connect it to the repository.
- 3) Add a PostgreSQL database plugin in the Railway dashboard
- 4) Export `SECRET_KEY` environment variable.
- 5) Railway identifies your Procfile and automatically builds and launches your app. and starts the application.
- 6) The `DATABASE_URL` is injected automatically by Railway.
- 7) On first request, `db.create_all()` creates users and predictions tables in PostgreSQL at the initial request.
- 8) Application is available at the domain provided by the Railway.

IX. DISCUSSION

A. Model Comparison

MuRIL is the best overall model yet it is also the heaviest: It has about 178 million parameters and needs a GPU for quick inference. The other end of the spectrum is Logistic Regression: little file size, loaded in milliseconds, and runs on the CPU. For a live web app with real users, response time matters. The current deployment is using lazy loading so that the first model that is requested is loaded into memory and kept there for future requests. We suggest Logistic Regression as the model for most users, since it achieves near MuRIL accuracy but with substantially faster reaction times. MuRIL is for those who demand the highest precision and are willing to wait a little longer. The BiLSTM falls in the middle but has more missed HOF postings than either of the two previous models. For content moderation use cases where false negatives (missed hate posts) are costly, the BiLSTM is less useful in practice.

B. Frontend Design Decisions

We decided to go with server-side rendering using Jinja2 templates instead of a separate frontend framework (React, Vue) on purpose. Server-side rendering is easier to deploy, doesn't have a separate build phase, and lets you maintain the whole program in one repository. The only client side Javascript is for the live prediction on dashboard, and the CSV export. These are tiny fetch based interactions that don't need a full frontend framework

C. Security

The application has several built-in security features:

- bcrypt hashing for passwords before storing. The plain text password is never saved or logged.
- All protected routes require login with Flask-Login decorators. Any prediction or history endpoint without a valid session will redirect to the login page.
- The `SECRET_KEY` is read from an environment variable and never committed to the repo.
- The input text is limited to 2,000 characters, so that requests do not get too long and slow down the server.
- SQLAlchemy's database queries automatically, preventing SQL injection attacks.

D. Limitations and Future Work

The biggest problem with the current setup is that without a GPU, inference using MuRIL is slow. MuRIL forecasts on the cloud deployment take several seconds every request as Railway's free tier does not have a GPU. This could be solved in future work by caching predictions for repeated inputs, utilizing a distilled transformer model of smaller size, or introducing a dedicated GPU instance. The hinglish capture of the mode is not upto the par whcih shows the low results across different models

The training data only contains HASOC 2019 and 2020 data. Future development could be training on newer datasets, adding support for more Indian languages , fine tuning the models on domain specific data like political discussion or news comments..

X. CONCLUSION

The project provided the full design of a multilingual hate speech detection web application. The system touches all levels of the stack from raw dataset to a real-world deployed service. We trained three machine learning models on the combined datasets of HASOC 2019 and 2020:

- **Logistic Regression** :quick, lightweight and competitive to the best model.
- **BiLSTM with Attention** :deep learning model using pretrained FastText embeddings, constrained by dataset size.
- **MuRIL Transformer**: highest accuracy achieved through large scale pre-training on Indian languages.

All three models are deployed in Railway with user authentication, a multi-page HTML interface, and a PostgreSQL database for the prediction history in a Flask web application. The project shows the power of combining excellent pre-processing, well-selected practicable designs, and clean web deployment might result in a viable a hate speech detection technology that is accurate, usable and maintainable.

REFERENCES

- [1] Z. Waseem and D. Hovy, "Hateful symbols or hateful people? Predictive features for hate speech detection on Twitter," in *Proc. NAACL Student Research Workshop*, 2016, pp. 88–93.
- [2] T. Davidson, D. Warmesley, M. Macy, and I. Weber, "Automated hate speech detection and the problem of offensive language," in *Proc. ICWSM*, vol. 11, 2017, pp. 512–515.
- [3] C. Nobata, J. Tetreault, A. Thomas, Y. Mehdad, and Y. Chang, "Abusive language detection in online user content," in *Proc. WWW*, 2016, pp. 145–153.
- [4] A. Schmidt and M. Wiegand, "A survey on hate speech detection using natural language processing," in *Proc. SocialNLP Workshop at EACL*, 2017, pp. 1–10.
- [5] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. ICLR*, 2015.
- [6] Z. Zhang, D. Robinson, and J. Tepper, "Detecting hate speech on Twitter using a convolution-GRU based deep neural network," in *Proc. ESWC*, 2018, pp. 745–760.
- [7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [8] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [9] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. ACL*, vol. 5, pp. 135–146, 2017.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [11] M. Mozafari, R. Farahbakhsh, and N. Crespi, "A BERT-based transfer learning approach for hate speech detection in online social media," in *Proc. Complex Networks*, 2020, pp. 928–940.
- [12] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [13] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. ACL*, 2020, pp. 8440–8451.
- [14] S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, "A survey of code-switched speech and language processing," *arXiv preprint arXiv:1904.00784*, 2019.
- [15] B. Patro, S. Bansal, and V. N. Balasubramanian, "Abusive language detection in Hinglish code-mixed data," in *Proc. CONSTRAINT Workshop at AAAI*, 2021.
- [16] M. Kula, M. Kocon, and M. Kajdanowicz, "Bert for hate speech detection," *arXiv preprint arXiv:2010.12472*, 2020.
- [17] S. Khanuja *et al.*, "MuRIL: Multilingual representations for Indian languages," *arXiv preprint arXiv:2103.10730*, 2021.
- [18] A. Velankar, P. Patil, and R. Joshi, "Hate and offensive speech detection in Hindi and Marathi," in *Proc. CONSTRAINT Workshop at ACL*, 2022.
- [19] T. Mandl *et al.*, "Overview of the HASOC track at FIRE 2019," in *Proc. FIRE*, 2019.
- [20] T. Mandl *et al.*, "Overview of the HASOC subtrack at FIRE 2020," in *Proc. FIRE*, 2020.
- [21] T. Ranasinghe, C. Orasan, and R. Mitkov, "Brums at HASOC 2019: Deep learning models for multilingual hate speech and offensive content identification," in *Proc. FIRE*, 2019.
- [22] M. Grinberg, *Flask Web Development*, 2nd ed. O'Reilly Media, 2018.